

UNITAT 8 · SETMANES 13–14

# Xarxa: els sòcols

📅 Setmanes 13–14 · dates per confirmar </> Laboratori TD7 (result + repte IA)

~ 2 animacions

MANUAL M8 · TEORIA DE LA UNITAT

## Xarxa: els sòcols

Lectura bàsica de la unitat 8 ( $\approx$  1 h 30 min). Les sortides són reals: programes del TD7 executats a la màquina del curs, sobre 127.0.0.1.

### CONTINGUT

- |    |                                     |    |                                  |
|----|-------------------------------------|----|----------------------------------|
| §1 | Un sòcol és un descriptor           | §4 | Models de servidor               |
| §2 | El cicle de vida d'una connexió TCP | §5 | Els datagrames i la xarxa d'avui |
| §3 | Adreces, ports i noms               |    |                                  |

### §1 Un sòcol és un descriptor

Un **sòcol** és l'extrem d'una comunicació. `socket(domain, type, protocol)` retorna un descriptor de fitxer, com `open` o `pipe`: es fa servir amb `read`, `write` i `close`, s'hereta amb `fork` i ocupa una entrada de la taula de descriptors. El *domini* diu quines adreces fa servir: `AF_UNIX` (un camí del sistema de fitxers, a la mateixa màquina), `AF_INET` (IPv4) o `AF_INET6` (IPv6). El *tipus* diu com es comunica:

- **Amb connexió** (`SOCK_STREAM`, TCP a Internet): s'estableix una connexió i després hi circula un **flux de bytes** fiable i ordenat, com per una canonada. És el telèfon de la diapositiva 181.
- **Sense connexió** (`SOCK_DGRAM`, UDP): cada enviament és un **datagrama** independent, que es pot perdre, duplicar o arribar desordenat. És el telegrama.

La diferència es pot mesurar. Tres `write` per TCP arriben en un sol `read` —TCP no sap res de missatges—, mentre que per UDP cada `recvfrom` retorna un datagrama sencer; i si el receptor no llegeix a temps, UDP perd dades sense que ningú rebi cap error, fins i tot a la mateixa màquina:

### Sortida real

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3 "two", recvfrom
#3: 5 "three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8 received, 9992
lost
kernel counter Udp RcvbufErrors grew by 9992
```

Conseqüència pràctica: per TCP, l'aplicació defineix on acaba cada missatge (HTTP/1 fa servir línies i `Content-Length`; el servidor d'eco del TD7, el final de línia). I com amb les canonades, `read` i `write` poden ser **parcials** (unitat 2): `write_all` fa un bucle fins que ho envia tot.

Practica-ho → A8.1, escenes 2 i 6 · TD7 · 4

## §2 El cicle de vida d'una connexió TCP

El **servidor** crea un sòcol, li dona una adreça i un port amb `bind` i crida `listen(fd, backlog)`: el sòcol passa a ser *passiu* (estat LISTEN) i només serveix per rebre connexions. El **client** crea el seu sòcol i crida `connect` amb l'adreça del servidor. Llavors els dos nuclis fan l'**encaixada de mans en tres passos**: SYN del client, SYN-ACK del servidor, ACK del client. El punt clau: **el nucli completa l'encaixada**, no l'aplicació. La connexió queda ESTABLISHED a la *cua d'acceptació* del sòcol d'escolta, i `accept` només la treu d'allà i retorna un **descriptor nou** per a aquest client (el d'escolta continua acceptant-ne d'altres).

El programa `tcp_states` fa els dos papers en un sol procés i demana al nucli l'estat de cada sòcol amb `getsockopt(TCP_INFO)`:

### Sortida real

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -          | server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED | server -
accept                      | listener LISTEN (queue 0/4) | client ESTABLISHED | server
ESTABLISHED
write/read                  | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2   | server
CLOSE_WAIT
server read                 | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED      | server
CLOSED
                           | the client's end moved to a TIME_WAIT entry of the kernel

(60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0        0      127.0.0.1:42772  127.0.0.1:PORT
```

El **tancament** és simètric i en dues meitats. Qui crida `close` primer (o `shutdown(fd, SHUT_WR)`) envia FIN i passa per `FIN_WAIT1` i `FIN_WAIT2`; l'altre costat rep el FIN, queda en `CLOSE_WAIT` i el seu `read` retorna 0 —final de fitxer, com amb una canonada sense escriptors—. Quan tanca al seu torn (`LAST_ACK`) i arriba l'últim ACK, el primer costat queda en **TIME\_WAIT** durant 2·MSL (60 s a Linux), per si l'últim ACK s'ha perdut i es repeteix el FIN. Aquesta entrada sobreviu al programa (`ss` la mostra) i és el motiu de `SO_REUSEADDR`: sense ell, tornar a engegar un servidor falla amb `EADDRINUSE`.

El **backlog** no limita quants clients atén el servidor, sinó quantes connexions ja establertes poden esperar `accept`. Linux n'admet `backlog + 1`; amb la cua plena descarta el SYN i el client el torna a enviar amb espera exponencial (1 s, 2 s, 4 s...):

### Sortida real

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5, SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8, SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```



#### Per als curiosos: les SYN cookies

Amb un backlog molt petit (`./backlog 2 8`) el nucli acaba responant amb **SYN cookies**, una defensa contra les inundacions de SYN que no guarda estat. Si l'ACK final arriba amb la cua d'acceptació plena, es perd: el client es creu connectat i el servidor no en sap res. Els comptadors de `nstat` ho mostren al `TD7 · 4`. Per això els servidors reals demanen un backlog gran (el límit és `net.core.somaxconn`).

Practica-ho → A8.1 cicle de vida d'una connexió · TD7 · 1 i 4

### §3 Adreces, ports i noms

Cada domini té la seva estructura d'adreça — `sockaddr_un` amb un camí de fins a 108 bytes, `sockaddr_in` amb una adreça IPv4 i un port, `sockaddr_in6` per a IPv6— i totes es passen a `bind`, `connect` o `accept` com a `struct sockaddr *` juntament amb la seva mida; `struct sockaddr_storage` és prou gran per a qualsevol. Els nombres de dins de l'estructura van en **ordre de xarxa** (*big-endian*): d'aquí `htons` per al port i `htonl` per a l'adreça. L'exemple de la diapositiva 187 se n'oblidava, i l'error es veu:

#### Sortida real

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000:          the kernel bound port 22555
sin_port = htons(7000):   the kernel bound port 7000
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
ip_unprivileged_port_start = 1024
sin_port = htons(80):     bind: Permission denied
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28, sockaddr_un 110, sockaddr_storage
128
```

En un x86-64, 7000 = 0x1b58 es guarda com els bytes 58 1b; llegits com a *big-endian* són 0x581b = 22555, i el servidor acaba escoltant en un altre port. La mateixa sortida mostra tres coses més: `getaddrinfo` converteix un nom i un servei (de `/etc/hosts` i `/etc/services`, o del DNS) en una llista d'adreces IPv4 i IPv6 a punt per fer servir —substitueix `gethostbyname`, que només coneixia IPv4, no era segur amb fils i va sortir de POSIX el 2008—; els ports per sota de 1024 requereixen privilegis (`CAP_NET_BIND_SERVICE`); i les mides reals de les estructures.

Un sòcol `AF_UNIX` no fa servir ni IP ni ports: la seva adreça és un **fitxer de tipus sòcol**. A més, el nucli diu al servidor qui hi ha a l'altre costat (`SO_PEERCRED`: PID i UID), cosa que fan servir systemd, D-Bus o `/var/run/docker.sock` per decidir què permeten:

#### Sortida real

```
$ ./local fso.sock
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path holds 108
bytes
the server asks the kernel who connected: pid = mine, uid 1000 (my user)
read 5 bytes; after close the file stays until unlink
still there after close: yes
```

Practica-ho → TD7 · 4, parts 5 i 6

## §4 Models de servidor

Un servidor passa la major part del temps **esperant**: que arribi un client ( `accept` ) o que aquest client envii alguna cosa ( `read` ). Com organitza aquestes esperes decideix qui espera qui:

- **Iteratiu**: un sol fil atén un client cada vegada. Si un client és lent, tots els altres esperen a la cua.
- **Un procés per client** ( `fork` després d' `accept` ): el fill atén i el pare torna a `accept` . Aïllats entre ells, però crear un procés costa diverses vegades més que un fil (mesurat al TD6 · 5) i cal evitar els zombis.
- **Un fil per client**: més barat i amb memòria compartida, però 10.000 connexions serien 10.000 fils amb les seves piles.
- **Grup de fils**: k fils creats a l'inici consumeixen connexions d'una cua limitada que omple el fil principal —el productor—consumidor de la unitat 7—. Recursos limitats, però k clients lents el saturen.
- **Bucle d'esdeveniments**: un fil pregunta al nucli amb `epoll_wait` quins sòcols ja tenen dades i només atén aquests, així que mai no es bloqueja amb un client lent. És la resposta al problema C10K i el model de nginx o Node.js; `io_uring` (Linux 5.1) porta la idea més enllà.

El TD7 implementa tots cinc a `http_server.c` i els mesura amb 32 clients, primer 1 i després 8 que triguen 1 s a enviar la petició:

### Sortida real

```
$ ./http_server -m iterative & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 981 ms, max 982 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 982 ms, max 983 ms
```

### Sortida real

```
$ ./http_server -m pool & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 1 ms, max 1 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 981 ms, max 981 ms
```

### Sortida real

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 1 ms, max 2 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 1 ms, max 2 ms
```

Practica-ho → A8.2 models de servidor · TD7 · 3 i 4

## §5 Els datagrames i la xarxa d'avui

En mode sense connexió no hi ha `listen` ni `accept`: el servidor lliga un port conegut i espera amb `recvfrom`, que retorna un datagrama i l'adreça de qui l'ha enviat; el client fa servir `sendto` amb l'adreça de destí (diapositives 194–197). També es pot fer `connect` d'un sòcol UDP per fixar el destí i fer servir `send` / `recv`. UDP és la base del DNS, de la veu i el vídeo en temps real i, des del 2021, de **QUIC** (RFC 9000), sobre el qual funciona **HTTP/3** (RFC 9114): la fiabilitat i el xifratge s'implementen a sobre, al programa i no al nucli.

Dos canvis més respecte del curs original. Gairebé tot el trànsit va xifrat amb **TLS** (https, 443): el sòcol porta bytes xifrats i una biblioteca (OpenSSL, GnuTLS...) fa la feina. I els serveis sense xifrar de la diapositiva 183 (`telnet`, `ftp`) s'han substituït per `ssh` i https.

Practica-ho → A8.1, escena 6 (UDP) · TD7 · 4